

Exercício 30 – O Patch de Sexta-Feira e o Fantasma do Drift

Estudo de Caso: Configuration Drift em Ambientes DevSecOps

1. A Armadilha da Boa Intenção (Configuration Drift)

O incidente ocorreu devido à criação de uma divergência entre a infraestrutura real e a infraestrutura declarada no código.

Na sexta-feira, o Líder de Segurança realizou alterações diretamente no ambiente de produção utilizando uma credencial de emergência. Embora a intenção fosse proteger a empresa contra uma vulnerabilidade crítica, as mudanças não foram registradas no repositório Git nem nos arquivos Terraform.

Essa situação é conhecida como **Configuration Drift**, que ocorre quando o estado real da infraestrutura deixa de corresponder ao estado descrito pela Infraestrutura como Código (Infrastructure as Code – IaC).

Quando Mariana aprovou uma simples alteração visual na segunda-feira, a pipeline executou normalmente o processo de provisionamento utilizando Terraform.

O Terraform identificou que a infraestrutura em execução estava diferente da configuração definida no código e realizou a sincronização automática para restaurar o estado declarado.

Como consequência:

- Regras de firewall foram revertidas;
- Configurações críticas foram sobrescritas;
- A comunicação entre serviços foi interrompida;
- O sistema ficou indisponível.

Portanto, a falha não foi causada pela alteração do front-end, mas pela existência de uma divergência entre o ambiente real e o código-fonte da infraestrutura.

2. A Falsa Sensação de DevSecOps

A ação da equipe de Segurança resolveu temporariamente o problema, porém violou princípios fundamentais de DevOps e DevSecOps.

Em um ambiente DevSecOps maduro, qualquer alteração de infraestrutura ou segurança deve ser realizada através do mesmo fluxo utilizado para mudanças de software.

O procedimento correto seria:

1. Criar uma branch de emergência;

2. Atualizar os arquivos Terraform;
3. Ajustar as regras de segurança no código;
4. Submeter as alterações para revisão;
5. Executar a pipeline automatizada;
6. Aplicar as mudanças por meio da Infraestrutura como Código.

Esse processo garantiria:

- Versionamento;
- Auditoria;
- Rastreabilidade;
- Reprodutibilidade;
- Consistência entre ambientes.

A segurança deve estar integrada ao pipeline e não ser aplicada por meio de intervenções manuais em produção.

3. Resiliência e Imutabilidade

Para evitar ocorrências semelhantes no futuro, a Fintech deve fortalecer seus mecanismos de governança, automação e controle.

Infraestrutura Imutável

Os recursos devem ser recriados a partir do código sempre que necessário, evitando modificações manuais diretas.

GitOps

O repositório Git deve ser considerado a única fonte oficial da verdade para toda a infraestrutura.

Detecção Automática de Drift

Ferramentas de verificação contínua devem identificar divergências entre o ambiente real e o código definido no repositório.

Controle de Acesso

O acesso direto à produção deve ser restrito e monitorado, permitindo alterações apenas por meio da pipeline oficial.

Auditoria e Compliance

Toda alteração deve gerar registros auditáveis, possibilitando rastreamento completo das ações realizadas.

Procedimentos de Emergência

Mesmo em situações críticas, deve existir um fluxo de mudança emergencial baseado em automação, garantindo rapidez sem comprometer a consistência do ambiente.

Dessa forma, a organização reduz riscos operacionais, aumenta a confiabilidade dos sistemas e fortalece os princípios de automação, segurança e resiliência que sustentam uma cultura DevOps moderna.